PROGRAM STUDI INFORMATIKA

Autentikasi, Otorisasi, dan Keamanan Dasar Web Service

Pertemuan 7 — Login, token JWT, role, pengantar OAuth2, validasi input, dan rate limiting

Autentikasi • Otorisasi • JWT • OAuth2 • Rate limiting

2 SKS • Semester Gasal 2026/2027

AGENDA

Agenda Hari Ini

01

Autentikasi vs otorisasi

Siapa kamu? Bolehkah kamu?

02

Konsep JWT

Struktur dan tanda tangan

03

Alur login-token

Dari login hingga akses

04

Pengantar OAuth2

Overview peran dan alur

05

Keamanan dasar

Validasi input, rate limiting

06

Praktikum

Login, role, dan uji client

CAPAIAN PEMBELAJARAN

Setelah Pertemuan Ini, Anda Mampu...

- Membedakan konsep autentikasi dan otorisasi, serta status 401 dan 403.
- Menerapkan autentikasi berbasis JWT pada REST API (login, token, endpoint terlindungi, role).
- Menerapkan praktik keamanan dasar: validasi input, prepared statement, rate limiting, hash password.
- Menjelaskan secara garis besar peran dan alur OAuth2.

MOTIVASI

API P6 Masih Terbuka untuk Siapa Saja

API Pertemuan 6 (tanpa autentikasi)

```
GET    /mhs       -> semua orang bisa baca
POST   /mhs       -> semua orang bisa tambah
DELETE /mhs/1     -> semua orang bisa HAPUS
GET    /log       -> semua orang lihat log
```



Risiko

Pencurian dan perusakan data
Tidak tahu siapa pelakunya
Penyalahgunaan (spam, brute force)
Pelanggaran privasi data mahasiswa

Siapa kamu?

AUTENTIKASI

Boleh melakukan ini?

OTORISASI

Seberapa sering?

RATE LIMITING

Inputnya aman?

VALIDASI

Siapa Kamu? Bolehkah Kamu?

Autentikasi (Authentication)

Membuktikan IDENTITAS.

"Saya budi" → dibuktikan dengan password / token.

Gagal: 401 Unauthorized

Otorisasi (Authorization)

Menentukan HAK AKSES.

"Budi boleh melihat, tetapi tidak boleh menghapus."

Gagal: 403 Forbidden

Contoh request	Kondisi	Status
`GET /mhs`	Tidak membawa token	401 — identitas tidak diketahui
`DELETE /mhs/1`	Token budi (role user)	403 — dikenali, tapi tidak berhak

Cara Membuktikan Identitas pada Web Service

Metode	Cara kerja	Catatan
API Key	Kunci tetap di header X-API-Key	Sederhana; identifikasi aplikasi, bukan pengguna
Basic Auth	username:password di tiap request	Wajib HTTPS; password terus berjalan
Session + Cookie	Server menyimpan sesi; client bawa cookie	Stateful; cocok untuk web biasa
Bearer Token (JWT)	Token bertanda tangan di header Authorization	Stateless; cocok untuk REST API (materi kita)
OAuth2	Izin terbatas dari pihak ketiga	Kerangka otorisasi; "Login dengan Google"

Pertemuan 5 memakai API key. Hari ini kita naik tingkat: login pengguna + token JWT + role.

Password Tidak Boleh Disimpan Apa Adanya

Cara simpan	Jika database bocor
Teks asli	Semua password terbaca
Enkripsi	Terbaca jika kunci ikut bocor
HASH + salt	Tidak bisa dikembalikan; harus ditebak satu per satu

Mengapa password_hash?

Algoritma lambat sengaja (bcrypt), salt acak otomatis: password sama menghasilkan hash berbeda.

PHP

```php
// saat membuat akun
$hash = password_hash("budi123",
PASSWORD_DEFAULT);
// -> $2y$10$Zw8rAKVFTx5wmo5Pzg5n...

// saat login
password_verify($input, $hash);   // true / false
```

Jangan buat algoritma sendiri; jangan pakai md5()/sha1() untuk password.

Session vs Token: Stateful vs Stateless

Session (stateful)

- Server menyimpan data sesi
- Client membawa ID sesi (cookie)
- Sulit diskalakan ke banyak server (sesi harus dibagi)
- Mudah dicabut: hapus sesi di server

Cocok: aplikasi web tradisional

Token JWT (stateless)

- Identitas ada DI DALAM token
- Server cukup memeriksa tanda tangan
- Mudah diskalakan, cocok untuk API/mobile
- Sulit dicabut sebelum kedaluwarsa

Cocok: REST API, microservices

Anatomi JSON Web Token

HEADER

```
{ "alg": "HS256",
  "typ": "JWT" }
```

Algoritma tanda tangan

PAYLOAD

```
{ "sub": 2,
  "username": "budi",
  "role": "user",
  "exp": 1792541910 }
```

Data (klaim) pengguna

SIGNATURE

```
HMACSHA256(
  header + "." + payload,
  KUNCI_RAHASIA )
```

Bukti keaslian

Bentuk akhir (dipisah titik)

```
eyJhbGciOiJIUzI1NiIs...  .  eyJzdWIiOjIsInVzZXJuYW1l...  .  Ecrz8TyP4sTk91_-EgLDM...
```

Isi Payload: Klaim (Claims)

Klaim	Arti
`sub`	subject: id pengguna
`username`	klaim kustom (kita tambah)
`role`	klaim kustom untuk otorisasi
`iat`	issued at: kapan dibuat
`exp`	expiration: kapan kedaluwarsa

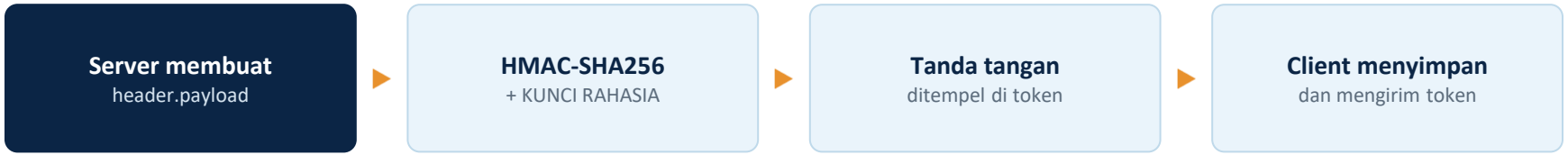


Bisa dibaca siapa saja

Base64 bukan enkripsi. Siapa pun yang menangkap token dapat membaca isinya.

Aturan: payload hanya berisi yang perlu untuk keputusan akses (id, username, role, exp). Keamanan JWT berasal dari TANDA TANGAN, bukan dari kerahasiaan isi.

Tanda Tangan Mencegah Pemalsuan



Percobaan penyerang	Yang terjadi di server	Hasil
Mengubah "role":"user" → "admin"	Tanda tangan dihitung ulang ≠ yang dikirim	401
Memakai kunci tebakan sendiri	Tanda tangan tidak cocok	401
Memakai token lama (exp lewat)	Pemeriksaan masa berlaku gagal	401
Token asal-asalan	Bukan 3 bagian / bukan JSON	401

Server menyatakan token sah hanya jika tanda tangan cocok DAN belum kedaluwarsa.

ALUR KERJA

Dari Login Sampai Mengakses Data

1

POST /login

username + password dikirim

2

Server memeriksa

password_verify di tabel users

3

200 + token

JWT dikirim ke client

4

GET /mhs + Bearer

Authorization: Bearer <token>

5

Server memeriksa token

sah → data | tidak → 401

Request yang membawa token

```
GET /mhs HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

jwt.php: Membuat dan Memeriksa Token

buat_token()

```
function buat_token($payload, $rahasia, $detik)
{
    $payload['iat'] = time();
    $payload['exp'] = time() + $detik;

    $h = base64url_encode(json_encode(
        ['alg'=>'HS256','typ'=>'JWT']));
    $p = base64url_encode(json_encode($payload));
    $t = base64url_encode(hash_hmac(
        'sha256', "$h.$p", $rahasia, true));
    return "$h.$p.$t";
}
```

cek_token()

```
function cek_token($token, $rahasia)
{
    $b = explode('.', $token);
    if (count($b) != 3) return null;
    list($h, $p, $t) = $b;

    $harus = base64url_encode(hash_hmac(
        'sha256', "$h.$p", $rahasia, true));
    if (!hash_equals($harus, $t)) return null;

    $data = json_decode(base64url_decode($p), true);
    if ($data['exp'] < time()) return null;
    return $data;
}
```

Autentikasi dan Otorisasi di Kode

fungsi.php

```
function wajib_login() {
    $token = ambil_token();
    $payload = $token ? cek_token($token) : null;
    if (!$payload) {
        kirim(['pesan'=>'Token tidak sah'], 401);
    }
    return $payload;
}
```

fungsi.php

```
function wajib_admin($pengguna) {
    if ($pengguna['role'] != 'admin') {
        kirim(['pesan'=>'Ditolak'], 403);
    }
}
```

index.php — dipakai pada endpoint

```
$pengguna = wajib_login();          // semua endpoint (kecuali /login)
elseif ($method == 'DELETE') { wajib_admin($pengguna); ... }   // hanya admin
```

Endpoint /login yang Aman

index.php — POST /login (ringkas)

```php
// 1) validasi input
if (empty($input['username']) || empty($input['password']))
    kirim([...], 422);

// 2) batasi percobaan gagal (rate limiting)
if ($gagal_semenit >= $batas_login) kirim([...], 429);

// 3) cari akun (prepared statement)
$stmt = $db->prepare("SELECT ... FROM users WHERE username = ?");

// 4) cek password terhadap hash
if (!$akun || !password_verify($pw, $akun['password']))
    kirim(['pesan' => 'Username atau password salah'], 401);

// 5) buat token
$token = buat_token([...], $jwt_secret, $jwt_berlaku);
```

Pesan seragam

Username salah dan password salah menghasilkan pesan yang sama.

Prepared statement

Input "' OR '1'='1" gagal menembus login.

Token berisi

sub, username, role, exp. Tanpa password.

Peta Hak Akses Endpoint

Method	URL	Akses	Gagal jika
POST	/login	Publik	401 salah | 422 kurang | 429 terlalu sering
GET	/me	Login	401 tanpa token
GET	/mhs, /mhs/{id}	Login	401 tanpa token
POST / PUT / DELETE	/mhs, /mhs/{id}	Admin	401 tanpa token | 403 role user
GET	/log	Admin	401 | 403

Akun latihan: admin / admin123 (role admin) dan budi / budi123 (role user). Hanya untuk praktikum!

Kelemahan JWT dan Cara Menyikapinya

Sulit dicabut

Token tetap sah sampai exp. Solusi: masa berlaku PENDEK + refresh token / daftar blokir.

Wajib HTTPS

Token dan password terlihat jika dikirim lewat HTTP biasa.

Simpan token

localStorage rentan XSS; cookie HttpOnly lebih aman tetapi perlu perlindungan CSRF.

Kunci rahasia

Panjang, acak, di environment variable. Bocor = semua token bisa dipalsukan.

OAuth2: Memberi Izin Tanpa Memberi Password

Resource Owner

Pengguna pemilik data

Client

Aplikasi yang meminta akses

Authorization Server

Memberi izin & menerbitkan token

Resource Server

API yang menyimpan data

OAuth2 adalah kerangka OTORISASI (delegasi izin), bukan protokol login. Untuk "siapa penggunanya" ditambahkan OpenID Connect (OIDC) di atas OAuth2. Token yang dipakai sering berupa JWT.

Alur Authorization Code (Disederhanakan)

1. Pengguna klik "Login dengan Google"
2. Diarahkan ke Authorization Server; pengguna login & menyetujui
3. Aplikasi menerima KODE sementara
4. Aplikasi menukar kode → ACCESS TOKEN
5. Aplikasi memanggil API dengan Bearer token

Istilah	Singkatnya
OAuth2	Delegasi izin
OIDC	Identitas (login)
JWT	Format token
Scope	Batas izin yang diminta

Header Authorization: Bearer yang kita pakai SAMA dengan OAuth2.

Validasi Input: Jangan Percaya Data dari Client

Ancaman / kesalahan	Pertahanan	Contoh di proyek
SQL Injection	Prepared statement (?)	WHERE username = ?
id berisi teks aneh	Validasi tipe	ctype_digit($id) → 400
Field kosong / salah tipe	Cek wajib + tipe	is_string, empty → 422
Body bukan JSON	Cek hasil json_decode	is_array → 400
XSS di tampilan client	Escape saat menampilkan	fungsi esc() di index.html
Kebocoran informasi	Pesan error umum	Detail SQL hanya di log

Validasi dilakukan di SERVER. Validasi di JavaScript client hanya untuk kenyamanan, mudah dilewati.

Rate Limiting: Membatasi Jumlah Request

fungsi.php — batasi_request()

```
$semenit_lalu = date('Y-m-d H:i:s', time() - 60);
$stmt = $db->prepare(
  "SELECT COUNT(*) FROM log_api WHERE ip = ? AND waktu >= ?");
$stmt->execute([$ip, $semenit_lalu]);

if ($stmt->fetchColumn() >= $batas_umum) {
    header('Retry-After: 60');
    kirim(['pesan' => 'Terlalu banyak request...'], 429);
}
```

Batas	Nilai
Login gagal	5 / menit / username
Request umum	120 / menit / IP
Status	429 Too Many Requests
Header	Retry-After: 60

- Hitung gagal login per (IP + username) agar akun asli tidak ikut terkunci oleh penyerang acak.
- Algoritma lain: fixed window (ini), sliding window, token bucket.

PRAKTIKUM

Praktikum: Menambah Autentikasi pada API Sederhana

- Impor mhs.sql versi P7 (tabel users + kolom username di log).
- Jalankan latihan/jwt_demo.php: baca isi, ubah, kedaluwarsa.
- Login di Postman, pasang Bearer token, uji 401 / 403 / 200.
- Jalankan test_client.php (28 pengecekan) dan client web.
- Uji rate limiting; baca kolom username di log_api.

Bahan praktikum

Modul Praktikum Pertemuan 7

Paket: api-mhs-auth.zip
server/ — API + JWT
client/ — penguji + web
latihan/ — demo JWT & rate limit

Akun: admin/admin123, budi/budi123

Tugas P7: Mengembangkan Autentikasi

- POST /register: username unik, password min. 8 karakter, disimpan dengan password_hash, role otomatis user.
- PUT /me/password: wajib mengirim password lama yang benar.
- Role baru "dosen": boleh mengubah data, tidak boleh menghapus.
- 3 query SQL analisis log: login gagal per username, IP terbanyak, jumlah 401/403/429 per hari.
- Perbarui test_client.php dengan minimal 8 pengecekan baru.

Tantangan: logout yang sungguh mencabut token (jti + blacklist), atau access token + refresh token.

RINGKASAN

Poin Penting Hari Ini

Authn vs Authz

401: siapa kamu? 403: bolehkah kamu?

JWT

header.payload.signature; tanda tangan mencegah pemalsuan

OAuth2

Delegasi izin tanpa membagikan password

Keamanan dasar

Hash password, validasi, prepared, rate limit, HTTPS

Prinsip: jangan pernah percaya input, selalu periksa identitas DAN hak akses di server.

Referensi

- RFC 7519 — JSON Web Token (JWT): rfc-editor.org/rfc/rfc7519
- RFC 6749 — The OAuth 2.0 Authorization Framework: rfc-editor.org/rfc/rfc6749
- RFC 6750 — OAuth 2.0 Bearer Token Usage: rfc-editor.org/rfc/rfc6750
- OWASP — Authentication Cheat Sheet & Password Storage Cheat Sheet: cheatsheetseries.owasp.org
- OWASP API Security Top 10: owasp.org/API-Security
- PHP Manual — password_hash: php.net/manual/en/function.password-hash.php
- jwt.io — Introduction to JSON Web Tokens: jwt.io/introduction
- MDN — HTTP authentication & status 401/403/429: developer.mozilla.org/en-US/docs/Web/HTTP